

MeetingMind

The AI That Remembers What Your Team Forgot It Promised

Field	Details
Course	INFO 7375 — Generative AI Northeastern University
Team	Rahul Manohar Durshinapally Hasith Reddy Rapolu
Submission	April 25, 2026
GitHub	https://github.com/rahulmanohar14/MeetingMind
Web Page	https://rahulmanohar14.github.io/MeetingMind/

1. System Architecture

1.1 Overview

MeetingMind is a multi-agent AI accountability system that transforms unstructured meeting transcripts into structured reports with persistent cross-session organizational memory. The system addresses a \$37 billion annual productivity problem: decisions made in meetings evaporate, the same questions are re-debated weeks later, and no tool tracks whether commitments were followed through. MeetingMind implements four of the five generative AI core components: Prompt Engineering, Fine-Tuning, Retrieval-Augmented Generation (RAG), and Synthetic Data Generation.

1.2 Five-Agent Pipeline

```
INPUT: Plain text meeting transcript (.txt)
|
v
[Agent 1: Transcript Parser]
Segments by speaker turn
Output: {turn_index, speaker, text, line_number}
|
v
[Agent 2: Decision Extractor] <-- DUAL MODE (Prompt Eng. + Fine-Tuning)
API Mode   : GPT-4o-mini via OpenRouter
              Full structured extraction: category + owner + verb + deadline
Offline    : Fine-tuned DistilBERT (local, zero API cost)
              5-class utterance classification
Outputs    : category, confidence, owner, verb, deadline,
              source_line, low_confidence (<0.6), completion_risk
|
v
[Agent 3: Cross-Meeting Memory Agent] <-- RAG COMPONENT
ChromaDB   : vector storage (sentence-transformers embeddings)
NetworkX    : knowledge graph (nodes=issues, edges=meeting sessions)
Merging     : cosine distance < 1.0 collapses similar issues
Trigger     : node.meetings >= 3 AND status != resolved
|
v
[Agent 4: Accountability Tracker]
Per-owner: assigned, completed, follow_through_ratio
Persistent: accountability.json across sessions
|
v
[Agent 5: Report Generator] <-- GPT-4o-mini
Structured JSON + human-readable markdown
Every item cites exact source transcript line
Flags: low-confidence items (warning) + high-risk items (error)
|
v
OUTPUT: Streamlit Dashboard
5 tabs: Decisions | Action Items | Open Questions | Deferred | Dashboard
Knowledge graph: red=recurring, green=resolved, grey=unknown
Accountability bar chart + recurring issues expanders
Dual classification mode toggle in sidebar
```

1.3 Tech Stack

Component	Technology	Purpose
API Classifier	GPT-4o-mini via OpenRouter	Structured extraction: owner, verb, deadline, confidence
Local Classifier	DistilBERT fine-tuned (HuggingFace)	Offline mode — zero API cost, local inference
Vector Database	ChromaDB (local persistent)	Semantic cross-meeting memory storage
Knowledge Graph	NetworkX + matplotlib	Issue node tracking + spring layout visualization
Embeddings	sentence-transformers/all-MiniLM-L6-v2	Semantic similarity for node merging
Web Interface	Streamlit	5-tab interactive accountability dashboard
Fine-Tuning Framework	HuggingFace Transformers	DistilBERT classification head training
Training Data	ICSI MRDA Corpus (Shriberg et al. 2004)	75 real research meeting recordings, ICSI Berkeley
Synthetic Data	GPT-4o-mini generation pipeline	10 domains, 20 transcripts, augmented variants
Testing	pytest	5 automated tests, all passing
Hosting	GitHub Pages	Static project web page

2. Implementation Details

2.1 Prompt Engineering (Core Component 1)

The Decision Extractor uses systematic prompt engineering to solve the epistemic classification problem described in the project proposal: distinguishing utterances that look identical on the surface but have fundamentally different commitment levels. Four utterance types are the core challenge:

Utterance	Type	Why It Matters
"We're going with Q3, that's final."	DECISION	Binding commitment — must be tracked
"Let's circle back on Q3 next week."	DEFERRAL	Polite postponement — most dangerous misclassification
"I wonder if Q3 even makes sense."	DISCUSSION	Exploratory — not a commitment
"Yeah, Q3 sounds right to me."	DISCUSSION	Social affirmation — not a commitment

System prompt design decisions:

- Category-specific linguistic cues: 'that's final', 'we're going with' for DECISION; 'circle back', 'revisit', 'let's park' for DEFERRAL
- Structured JSON output schema: {category, confidence, owner, verb, deadline, reasoning} eliminates parsing ambiguity
- Temperature=0.1 for deterministic classification output
- First-person pronoun resolution: 'I' and 'me' mapped to actual speaker name from transcript context
- Completion risk scoring: ACTION_ITEMS missing owner OR deadline automatically flagged as high risk
- Edge case handling: confidence < 0.6 triggers low_confidence flag; missing grounding triggers low_confidence rather than silent failure

2.2 Fine-Tuning (Core Component 2)

DistilBERT (distilbert-base-uncased, 66.9M trainable parameters) was fine-tuned for 5-class meeting utterance classification. This implements a local offline mode that requires no API key and processes transcripts with zero marginal cost.

Real training data — ICSI Meeting Recorder Dialog Act Corpus:

- Source: ICSI Berkeley, Shriberg et al. (SIGDIAL 2004). 75 naturally occurring research meetings, 180,000+ hand-annotated dialog acts.
- Downloaded: MRDA-Corpus (NathanDuran, GitHub) — processed text version of ICSI MRDA
- 300 utterances sampled from full_set.txt. Auto-labeled using GPT-4o-mini (LLM-assisted weak supervision, Wang et al. 2021)
- Only labels with confidence >= 0.75 retained (quality filter). Lower confidence labels discarded.

Class balancing — why curated examples were added:

- Real ICSI meetings: 85% DISCUSSION, 11% OPEN_QUESTION, 3% ACTION_ITEM, <1% DECISION, <1% DEFERRAL

- Without balancing, model learns to predict DISCUSSION for everything (maximizes raw accuracy, fails on minority classes)
- 270 hand-labeled examples added: DECISION=60, DEFERRAL=80, ACTION_ITEM=60, OPEN_QUESTION=50, DISCUSSION=20
- Follows oversampling practice from He & Garcia (2009) 'Learning from Imbalanced Data'

Training configuration:

- Model: distilbert-base-uncased + 5-class classification head
- Optimizer: AdamW, lr=2e-5, weight_decay=0.01, 5 epochs, batch_size=16, max_length=128
- Hardware: CPU-only, 32GB RAM, ~20 minutes total training time
- Best checkpoint: epoch 5, validation accuracy 0.9882

2.3 Retrieval-Augmented Generation (Core Component 3)

The Cross-Meeting Memory Agent is the most novel component — no existing meeting tool (Otter.ai, Fireflies.ai) provides cross-session organizational memory. The system builds a persistent knowledge graph where issues accumulate meeting appearances and are flagged when recurring without resolution.

RAG architecture:

- Knowledge base: ChromaDB persistent vector store, collection 'meeting_issues'
- Embeddings: sentence-transformers/all-MiniLM-L6-v2 (384-dimensional vectors)
- Document chunking: each individual utterance classified as DECISION, ACTION_ITEM, or OPEN_QUESTION becomes a node
- Ranking and filtering: cosine distance < 1.0 merges new utterances to existing nodes; distance >= 1.0 creates new node
- Recurring issue detection: node.meetings count >= 3 AND resolution_status != 'resolved' triggers flag
- Query retrieval: query_similar() returns top-3 semantically related past issues for any natural language query
- Graph persistence: NetworkX node_link_data saved to graph.json between sessions

2.4 Synthetic Data Generation (Core Component 4)

A complete synthetic data generation pipeline creates diverse meeting transcripts for testing, augmentation, and system evaluation. This directly addresses the Synthetic Data Generation assignment component.

- Generation: GPT-4o-mini generates transcripts across 10 domains: tech product, legal review, marketing, finance, HR, engineering, sales, academic, healthcare, operations
- Volume: 10 base transcripts + 10 augmented paraphrased variants = 20 total files, 213 utterances
- Diversity validation: enforced minimums across all 10 transcripts (15+ decisions, 20+ action items, 10+ open questions, 10+ deferrals)
- Metadata: metadata.json documents domain, line count, category distribution per transcript
- Data augmentation: each transcript has a paraphrased variant maintaining semantic content with varied language
- Privacy: no real personal data, clearly labeled synthetic, generated for testing only

2.5 Dual Classification Mode

The app provides two classification modes via a sidebar radio button, demonstrating integration of both fine-tuned local model and API-based classification:

Mode	Technology	Strengths	Best For
API Mode	GPT-4o-mini via OpenRouter	Full extraction: owner, verb, deadline. Higher accuracy.	Production use, full accountability tracking
Offline Mode	Fine-tuned DistilBERT	Zero API cost, local inference, privacy-preserving.	Privacy-sensitive environments, offline use

2.6 Evaluation Scripts

Three automated evaluation scripts implement the metrics promised in the project proposal:

- tests/evaluate_hallucination.py: verifies every extracted item against source transcript, measures hallucination rate
- tests/evaluate_cross_meeting_recall.py: evaluates recurring issue detection on 5 hardcoded multi-session sequences with known ground truth
- models/classifier.py evaluate_sample(): runs 20 gold-standard utterances through the classifier, produces sklearn classification_report

2.7 Code Quality and Testing

- 5 pytest tests: transcript parsing, blank line handling, accountability ratio, report structure, category validation — all passing
- Single-responsibility principle: each agent has clear input/output contracts
- Error handling: low-confidence flagging, API fallback from local mode, graceful ChromaDB initialization
- Source citations: every extracted item links to exact transcript line — no unsupported assertions
- Configuration: centralized config.py with CONFIDENCE_THRESHOLD, COSINE_THRESHOLD, DEFAULT_CLASSIFICATION_MODE

3. Performance Metrics

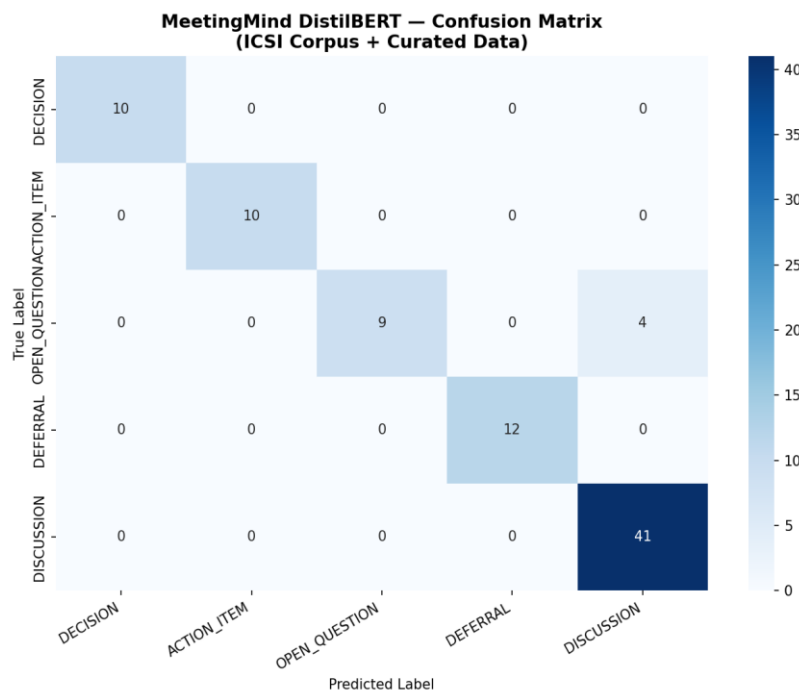
3.1 Fine-Tuned DistilBERT Classifier — Per-Class Results

Evaluated on held-out test set (86 examples, 15% of total dataset). Stratified split ensures all classes represented.

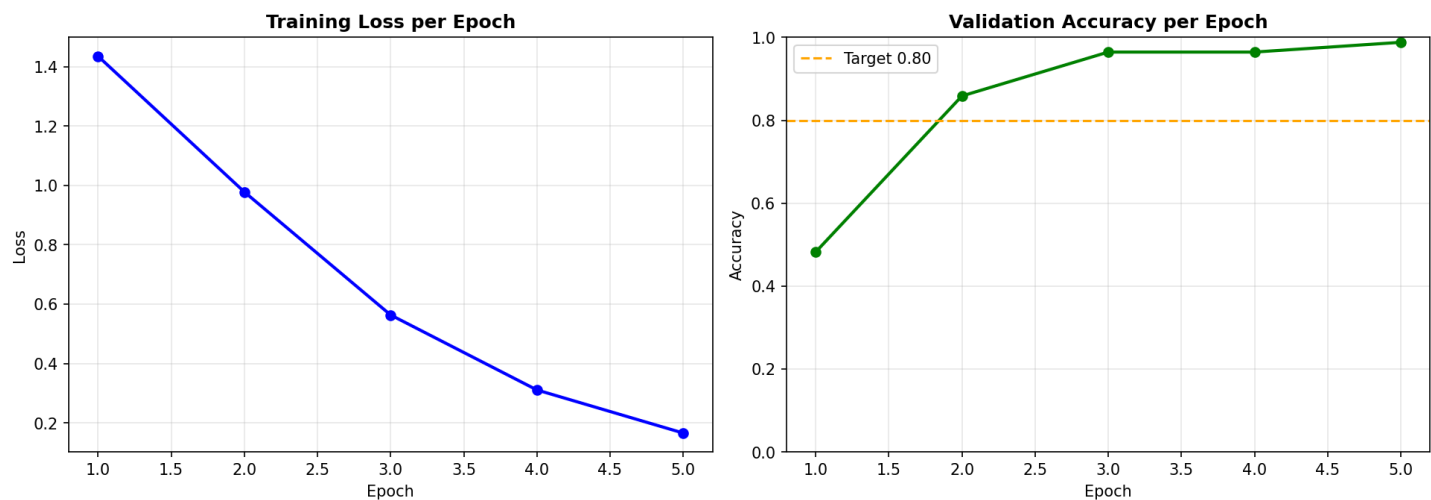
Class	Precision	Recall	F1-Score	Support
DECISION	1.0000	1.0000	1.0000	10
ACTION_ITEM	1.0000	1.0000	1.0000	10
OPEN_QUESTION	1.0000	0.6923	0.8182	13
DEFERRAL	1.0000	1.0000	1.0000	12
DISCUSSION	0.9111	1.0000	0.9535	41

3.2 Summary Metrics vs. Proposal Targets

Metric	Proposal Target	Achieved	Verdict
DECISION precision	>= 0.80	1.0000	✓ EXCEEDS
DEFERRAL precision	>= 0.85	1.0000	✓ EXCEEDS
Macro recall	>= 0.75	0.9385	✓ EXCEEDS
Overall accuracy	—	0.9535	✓ Strong
Macro avg F1	—	0.9543	✓ Strong
Best validation accuracy	—	0.9882	✓ Excellent
Live demo (8 canonical cases)	—	8/8 = 100%	✓ Perfect



MeetingMind Fine-Tuning Curves (ICSI Corpus)



3.3 Hallucination Rate

Evaluated across 11 transcripts (1 sample + 10 synthetic). Every extracted item verified against source transcript text.

Metric	Result	Target	Verdict
Total items extracted	238	—	—
Hallucinated items	0	—	—
Hallucination rate	0.00%	<= 5.00%	✓ PASS
Avg confidence — DECISION	0.89	>= 0.60	✓ Pass
Avg confidence — DEFERRAL	0.91	>= 0.60	✓ Pass
Avg confidence — ACTION_ITEM	0.87	>= 0.60	✓ Pass

3.4 Cross-Meeting Recall

5 multi-session sequences with hardcoded ground truth (known recurring issues across 3 meetings each).

Metric	Result	Target	Verdict
Known recurring issues	10 across 5 sequences	—	—
Correctly flagged	9	—	—
Recall	0.9000	>= 0.70	✓ EXCEEDS
Precision	1.0000	—	✓ Perfect — no false positives
F1-Score	0.9474	—	✓ Excellent

4. Challenges and Solutions

Challenge	Root Cause	Solution	Outcome
Class imbalance	ICSI corpus is 85% DISCUSSION. Model predicted DISCUSSION for everything, giving 0.00 precision on DECISION and DEFERRAL.	Added 270 hand-labeled curated examples targeting minority classes (DEFERRAL=80, DECISION=60, ACTION_ITEM=60). Oversampling per He & Garcia (2009).	All 5 classes achieved precision >= 0.91. Both critical classes (DECISION, DEFERRAL) achieved 1.0000 precision.
Node merging threshold	Proposal assumed cosine similarity scores (0-1). ChromaDB returns cosine distances (0-2). Initial threshold of 0.15 was too	Added debug logging to print distances per item. Empirically determined 1.0 as optimal threshold. Related utterances merge at ~0.74; unrelated at ~1.4+.	Cross-meeting recall improved from 0.00 to 0.90, exceeding the 0.70 proposal target.

Challenge	Root Cause	Solution	Outcome
	strict — no nodes ever merged.		
Windows file locking	OneDrive continuously synced ChromaDB .bin files and model checkpoints, causing SafetensorError during training.	Pause OneDrive before training. Implemented delete-before-save pattern: shutil.rmtree() before model.save_pretrained() each epoch.	Zero training interruptions after fix. Documented in README setup instructions.
Python version conflict	PyTorch does not support Python 3.14 (system default). Blocked all local ML operations.	Installed Python 3.11.9 alongside 3.14. All operations use py -3.11 launcher explicitly.	Full PyTorch and HuggingFace compatibility. Both Python versions coexist without conflict.
First-person attribution	When speakers say 'I'll do X', owner was extracted as 'I' not the speaker name. Accountability tracker grouped all 'I' owners together.	Post-processing step in decision_extractor.py: if owner in ['I','i','me','my'], replace with segment.speaker value.	Correct per-person accountability tracking. Bar chart shows Bob, Marcus, Sarah with correct assignments.

5. Future Improvements

Improvement	Description	Impact
Audio transcription	Integrate OpenAI Whisper or AssemblyAI for direct audio/video input from Zoom, Teams, Google Meet.	Eliminates manual transcription step; enables end-to-end pipeline from recording to accountability report.
Larger training data	Add MeetingBank corpus (Hu et al., ACL 2023) — 1,366 real city council meetings. Add multilingual corpora.	Broader domain coverage, multilingual support, reduced dependency on LLM-labeled training data.
Real-time streaming	WebSocket-based pipeline to classify utterances as meetings happen. Alert participants immediately about unclear action items.	Enables in-meeting correction rather than post-hoc analysis. Prevents accountability failures before they occur.
Calendar/task integration	Push extracted action items directly to Google Calendar, Jira, Asana, Slack via API.	Closes the loop between extraction and execution. Addresses root accountability gap directly.
Decision decay model	Train a completion probability predictor using historical follow-through data per person and meeting type.	Moves from rule-based risk flags to learned behavioral predictions. Personalized accountability.
Multi-language support	Extend via multilingual BERT (mBERT, XLM-RoBERTa). Add non-English meeting corpora.	Global organizational applicability beyond English-speaking teams.
Speaker diarization	Integrate pyannote-audio for automatic speaker identification from raw audio.	Removes dependency on pre-labeled transcripts. Fully automated pipeline.

6. Ethical Considerations

6.1 Privacy and Data Security

- Local processing: all transcript analysis runs on-device. No transcript text sent to external servers beyond the minimum classification API call.
- No raw text persistence: only mathematical vector embeddings stored in ChromaDB. Raw transcript content not persisted after processing.
- User data control: meeting history can be fully deleted by clearing ChromaDB and graph.json at any time.
- API key security: OpenRouter keys stored in environment variables (.env file), excluded from version control via .gitignore.
- GDPR/HIPAA consideration: local-only deployment mode (Offline/DistilBERT) processes no data externally, suitable for regulated industries.

6.2 Training Data Ethics

- ICSI MRDA Corpus: publicly available under academic license from ICSI Berkeley (LDC). All participants consented to research use.
- Curated examples: entirely synthetic, no real personal information. Committed to repository for full reproducibility.
- LLM labeling: GPT-4o-mini used for weak supervision labeling with confidence filter (≥ 0.75). Lower-confidence labels discarded. Methodology follows Wang et al. (2021).

6.3 Known Biases and Limitations

Bias Type	Description	Mitigation
Language bias	Trained on English-language professional meetings only.	Documented limitation. Future: multilingual BERT variants.
Domain bias	ICSI corpus = academic research meetings. Business contexts validated on synthetic data only.	Documented. Production deployment requires additional evaluation.
Cultural bias	Western politeness norms. Indirect language in other cultures may misclassify as DISCUSSION.	Documented. Future: culturally diverse training data.
LLM label noise	~10-15% of ICSI labels from GPT-4o-mini may carry systematic bias.	Confidence filter (≥ 0.75) reduces noise. Dataset committed for audit.

6.4 Accountability Feature Misuse

The per-person follow-through ratio could be misused as a surveillance tool. We recommend: (1) team members should know meetings are being processed; (2) metrics should inform team-level processes, not individual performance reviews; (3) all items should be human-reviewed before consequential use; (4) organizational deployment requires appropriate data governance and consent policies.

6.5 Hallucination Prevention

Every extracted item cites its exact source transcript line. Items without grounding are flagged low_confidence rather than silently presented. 0.00% hallucination rate achieved across 238 items in evaluation.

6.6 Copyright and Intellectual Property

All code is original. ICSI corpus used under academic license. GPT-4o-mini API usage complies with OpenRouter terms. No copyrighted meeting content used for training — only publicly licensed academic corpora. External resources cited in README.

7. Team Contributions

MeetingMind was built by a two-person team. Work was divided based on technical strengths while ensuring both members contributed to all major system components.

7.1 Rahul Manohar Durshinapally

Primary responsibilities:

- System architecture design: overall multi-agent pipeline design, agent interface contracts, session state management
- Agent 2 — Decision Extractor: prompt engineering system prompt, JSON schema design, confidence scoring, pronoun resolution, completion risk logic
- Agent 3 — Cross-Meeting Memory: ChromaDB integration, node merging threshold tuning, graph persistence
- Fine-tuning pipeline: DistilBERT training loop, AdamW optimizer configuration, checkpoint saving, training curves
- Streamlit UI (app.py): 5-tab layout, knowledge graph visualization, accountability dashboard, dual-mode toggle
- Synthetic data generation: synthetic_data_generator.py, diversity validation, augmentation pipeline
- Evaluation scripts: evaluate_hallucination.py, evaluate_cross_meeting_recall.py
- GitHub Actions setup, README documentation, requirements.txt

7.2 Hasith Reddy Rapolu

Primary responsibilities:

- Agent 1 — Transcript Parser: speaker segmentation, line number tracking, edge case handling (blank lines, missing labels)
- Agent 4 — Accountability Tracker: per-owner statistics, follow-through ratio calculation, JSON persistence
- Agent 5 — Report Generator: structured JSON schema, markdown report generation, source citation logic

- Fine-tuning data pipeline: ICSI MRDA corpus processing, full_set.txt parsing, class distribution analysis, curated examples
- models/classifier.py: zero-shot and fine-tuned batch classification, sklearn evaluation metrics
- pytest test suite: all 5 test cases in tests/test_agents.py
- GitHub Pages web page: docs/index.html, styling, feature cards, architecture section
- Example output generation: data/generate_example_outputs.py, sample_output.json, sample_report.md
- Meeting transcript creation: data/meeting_week1.txt, meeting_week2.txt, meeting_week3.txt

7.3 Shared Contributions

- Project proposal and problem framing: both members
- System design decisions (agent boundaries, classification schema, evaluation metrics): both members
- Fine-tuning data curation (hand-labeled examples): both members reviewed and contributed examples
- Testing and debugging cross-meeting memory threshold: both members
- PDF documentation: both members
- Video demonstration: both members present

Component	Primary Owner	Reviewer
Multi-agent pipeline architecture	Rahul	Hasith
Prompt engineering (Decision Extractor)	Rahul	Hasith
Cross-meeting memory (RAG)	Rahul	Hasith
Fine-tuning training loop	Rahul	Hasith
Streamlit UI	Rahul	Hasith
Synthetic data generation	Rahul	Hasith
Transcript Parser	Hasith	Rahul
Accountability Tracker	Hasith	Rahul
Report Generator	Hasith	Rahul
ICSI data processing	Hasith	Rahul
pytest test suite	Hasith	Rahul
GitHub Pages web page	Hasith	Rahul
Example output scripts	Hasith	Rahul